SS2 DIGITAL TECHNOLOGIES LESSON NOTES (SECOND TERM)

WEEK 1: Introduction to Algorithms and Flowcharts

Topic: Introduction to Algorithms and Flowcharts

Introduction:
Before a single line of code can be written, a programmer must have a clear, logical plan. This plan is known as an algorithm. This week, we will learn how to define these plans using plain language, pseudocode, and visual diagrams called flowcharts.

Subtopic 1: Understanding Algorithms

- **Definition:** An algorithm is a finite, step-by-step, unambiguous set of instructions designed to solve a specific problem or perform a specific task.
- **The "Recipe" Analogy:** A good algorithm is like a cooking recipe. It has:
  - **Inputs:** Ingredients (e.g., flour, eggs).
  - **Process:** A set of instructions (e.g., "mix flour and eggs").
  - **Output:** A finished product (e.g., a cake).
- **Properties of a Good Algorithm:**

1. **Finite:** It must end after a specific number of steps. It cannot run forever.
2. **Unambiguous:** Each instruction must be perfectly clear, with only one possible interpretation. A computer cannot "guess" your meaning.
3. **Input:** It has zero or more well-defined inputs (the data it works with).
4. **Output:** It produces one or more well-defined outputs (the result).

5. **Effective:** Every instruction must be basic enough to be carried out, in principle, by a person using only paper and pencil.

- **Representing Algorithms with Pseudocode:**

  - **Definition:** Pseudocode (meaning "false code") is a way of writing an algorithm using plain English-like statements before translating it into a specific programming language.
  - **Purpose:** It allows the programmer to focus on the logic of the problem without worrying about the strict grammar rules (syntax) of a language like Python.
  - **Example: Algorithm to find the average of three numbers (Pseudocode):**

1. START
2. GET number1
3. GET number2
4. GET number3
5. SET total = number1 + number2 + number3
6. SET average = total / 3
7. DISPLAY average
8. END

PETOA GROUP OF SCHOOLS

Subtopic 2: Visualizing Logic with Flowcharts

- **Definition:** A flowchart is a visual, diagrammatic representation of an algorithm. It uses a set of standard symbols connected by arrows to show the sequence of operations.
- **Advantages:** Makes complex logic easy to understand, helps find errors (bugs), and serves as a "blueprint" for the program.
- **Standard Flowchart Symbols:**

1. **Terminator (Oval):** Represents the **START** and **END** points.
2. **Process (Rectangle):** Represents an action or calculation (e.g., `SET total = a + b`).
3. **Input/Output (Parallelogram):** Represents getting data (`GET number`) or displaying data (`DISPLAY total`).
4. **Decision (Diamond):** Represents a question or a branching point (e.g., `Is number > 10?`). It has one input and two or more outputs (e.g., Yes/No).

5. **Flow Lines (Arrows):** Connect the symbols and show the direction of logic.

Evaluation Questions:

1. What is an algorithm? Explain two important properties of a good algorithm.
2. What is pseudocode and what is its primary advantage for a programmer?
3. What is a flowchart?
4. In a flowchart, what is the difference between the "Process" symbol and the "Decision" symbol?
5. Write the pseudocode for an algorithm that takes a student's score, checks if it is 50 or above, and then displays either "Pass" or "Fail".

WEEK 2: Introduction to Python and JavaScript

Topic: Introduction to Python and JavaScript

Introduction:
Now that we understand what an algorithm is, we must learn how to translate our logical plans into a language a computer can understand. This week, we introduce two of the most popular programming languages: Python and JavaScript.

Subtopic 1: Programming Language Hierarchy

- **Low-Level Languages:** These are close to the computer's hardware.
  - **Machine Code (1st Gen):** The only language a CPU understands. It is a sequence of binary digits (1s and 0s) and is unreadable by humans.
  - **Assembly Language (2nd Gen):** Uses short codes (mnemonics) like `ADD` and `MOV`. It's easier for humans than machine code but still tied to the hardware.

- **High-Level Languages (3rd Gen):** Languages like Python and JavaScript that use English-like syntax and mathematical notation. They are easy for humans to read and write.
  - **Abstraction:** They "hide" the complex details of the hardware, allowing programmers to focus on solving the problem.
- **Compilers vs. Interpreters:** High-level code must be translated into low-level machine code.
  - **Compiler:** Reads the entire program at once, translates it into an executable file, and then runs it. (e.g., C++, Java).
  - **Interpreter:** Reads and executes the program one line at a time. (e.g., Python, JavaScript). This makes them easier to debug.

Subtopic 2: Introduction to Python

- **History:** Created by Guido van Rossum and first released in 1991.
- **Why is Python so popular?**

1. **Readability:** Its syntax is clean, simple, and almost like writing English. It uses indentation to define code blocks.
2. **Versatility:** It is a "general-purpose" language used for web development, data science, AI, automation, and more.
3. **Huge Community:** A massive global community means infinite tutorials and support.

- **"Hello, World!" in Python:**

```python
# This is a comment in Python
print("Hello, World!")
```

Subtopic 3: Introduction to JavaScript (JS)

- **History:** Created by Brendan Eich in 1995.
- **Purpose:** JavaScript is one of the three core technologies of the World Wide Web.

- **Front-End (Client-Side):** It is the only language that runs natively inside a web browser. It's used to make websites interactive (e.g., image sliders, pop-up forms).
- **Back-End (Server-Side):** With Node.js, JavaScript can also be used to write server-side code.

- **"Hello, World!" in JavaScript:**

```javascript
// This is a single-line comment in JavaScript
console.log("Hello, World!"); // For the browser console
// To show an alert on a webpage:
// alert("Hello, World!");
```

Subtopic 4: Key Differences (Python vs. JavaScript)

- **Primary Use:** Python is used for back-end, data science, AI. JavaScript is used for front-end web development and back-end (Node.js).
- **Syntax:** Python uses clear, English-like syntax. JavaScript uses C-style syntax with curly braces `{}`.
- **Code Blocks:** Python uses indentation. JavaScript uses curly braces `{}`.

Evaluation Questions:

1. Differentiate between a High-Level Language and a Low-Level Language.
2. Explain the fundamental difference between a Compiler and an Interpreter. Which one is Python?
3. What is the primary role of JavaScript in web development?
4. What is the major syntactical difference between how Python defines a code block and how JavaScript defines one?
5. Write the "Hello, World!" code for both Python and JavaScript.

WEEK 3: Variables, Data Types, and Operators

Topic: Variables, Data Types, and Operators

Introduction:

A program is simply the manipulation of data. This week, we learn how to store data (in variables), the different types of data (data types), and how to manipulate that data (with operators). We will use Python for most examples.

Subtopic 1: Variables

- **The "Box" Analogy:** A variable is a named storage location in the computer's memory where you can store a piece of data. The variable name is the label on the box.
- **Assignment:** The = sign is the "assignment operator." It means "put the value on the right into the box on the left."

python

```
x = 10        # 'x' is the variable name, '10' is the value
name = "Bello" # Storing text
y = 5
sum = x + y   # Using variables in calculations
print(sum)    # Output: 15
```

- **Naming Conventions (Rules):**

1. Must start with a letter or an underscore (_).
2. Cannot start with a number.
3. Can only contain letters, numbers, and underscores.
4. Names are case-sensitive (`age` is different from `Age`).
5. **Best Practice (Python):** Use `snake_case` (all lowercase with underscores)

   e.g., `first_name`, `total_score`.

Subtopic 2: Fundamental Data Types

- **Definition:** A data type tells the computer how to interpret a value. It defines what kind of data a variable holds.
- **Common Data Types in Python:**

1. **int (Integer):** Whole numbers (e.g., `10`, `-5`, `0`).
2. **float (Floating Point):** Numbers with a decimal (e.g., `10.5`, `-3.14`).
3. **str (String):** A sequence of characters (text) in quotes (e.g., `"Hello"`, `'Nigeria'`).
4. **bool (Boolean):** Represents truth values: `True` or `False`.

- **Dynamic Typing:** In Python, you don't have to declare the type. The interpreter figures it out automatically.

Subtopic 3: Data Type Conversion (Casting)

- **The Problem:** The `input()` function in Python always returns a **string**. If you ask for a user's age and they type `20`, you get the string `"20"`, not the number `20`.
- **The Solution (Casting):** You must convert the string to a number.

python

```python
age_str = input("Enter your age: ") # age_str is "20" (a string)
age_int = int(age_str)              # Convert "20" to 20 (an integer)
new_age = age_int + 1
print(new_age)                      # Output: 21
```

- **Casting Functions:** `int()`, `float()`, `str()`.

Subtopic 4: Operators

1. **Arithmetic Operators:** For math (`+`, `-`, `*`, `/`, `//` (floor division), `%` (modulo/remainder), `**` (exponent)).
2. **Comparison Operators:** Compare two values, resulting in `True` or `False` (`==`, `!=`, `>`, `<`, `>=`, `<=`).
3. **Logical Operators:** Combine multiple Boolean expressions (`and`, `or`, `not`).

Evaluation Questions:

1. What is a variable? Explain the "box" analogy.
2. List the four main data types in Python with an example for each.

PETOA GROUP OF SCHOOLS

3. Why is type casting (e.g., using `int()`) necessary when using the `input()` function in Python for calculations?
4. What is the difference between the `/` operator and the `%` operator in Python?
5. What is the result of `(5 > 3) and (10 == 10)`? What is the result of `(5 < 3) or (10 != 10)`?

WEEK 4: Control Structures (If-Else, Loops, and Functions)

Topic: Control Structures: If-Else, Loops, and Functions

Introduction:
This week, we learn how to control the flow of a program. Control structures allow us to make decisions (running code only if a condition is met) and perform repetition (running code over and over).

Subtopic 1: Conditional Statements (If, Elif, Else)

- **Concept:** Conditionals allow your program to make decisions and execute different blocks of code based on whether a condition is `True` or `False`.
- **The `if` Statement (One Choice):**
  python
  ```python
  age = 20
  if age >= 18:
      print("You are eligible to vote.") # This runs only if condition is True
  print("Program finished.") # This always runs
  ```
- **The `if-else` Statement (Two Choices):**
  python
  ```python
  score = 45
  if score >= 50:
      print("You Passed.")
  else:
  ```

```
    print("You Failed.") # This runs only if condition is False
```

- **The `if-elif-else` Ladder (Multiple Choices):** `elif` is short for "else if".

  python

  ```
  score = 75
  if score >= 70:
      print("Grade: A")
  elif score >= 60:
      print("Grade: B")
  elif score >= 50:
      print("Grade: C")
  else:
      print("Grade: F")
  ```

- **Python's Indentation:** Python uses indentation to define a code block. This is a strict rule.

  Subtopic 2: Repetition (Loops)

- **Concept:** Loops allow you to repeat a block of code multiple times.
- **The `while` Loop:** Repeats as long as a condition remains `True`. It requires a counter variable.

  python

  ```
  count = 1           # 1. Initialize counter
  while count <= 5:   # 2. Set condition
      print(count)
      count = count + 1 # 3. Update counter (CRITICAL to avoid infinite loop)
  print("Loop finished.")
  ```

- **The `for` Loop:** The "counting loop," used to iterate over a sequence like a `range()` of numbers.

  python

  ```
  for i in range(1, 6): # range(start, stop_before) prints 1 to 5
      print(i)
  ```

Subtopic 3: Introduction to Functions

- **Concept:** A function is a named, reusable block of code that performs a specific task.
- **Why Use Functions? (The D.R.Y. Principle):**
  - **D.R.Y.** = **Don't Repeat Yourself.**
  - If you find yourself writing the same code in multiple places, put that code into a function and "call" it when needed. This makes code organized, readable, and easier to update.
- **Defining and Calling a Function:**

python

```python
# 1. DEFINING the function (the blueprint)
def greet_user():
    print("Hello, welcome!")
    print("We are glad to have you.")

# 2. CALLING the function (to run the code)
greet_user()
greet_user() # You can call it many times!
```

- **Functions with Parameters (Inputs):**

python

```python
def greet_by_name(name): # 'name' is a parameter
    print("Hello, " + name + "! Welcome.")

greet_by_name("Aisha")   # "Aisha" is the argument
greet_by_name("Chinedu")
```

- **Functions that Return a Value (Outputs):**

python

```python
def add_numbers(num1, num2):
    total = num1 + num2
    return total # Sends the value back

sum_result = add_numbers(5, 3) # Call the function and store the result
```

PETOA GROUP OF SCHOOLS

```
print(sum_result) # Output: 8
```

Evaluation Questions:

1. What is the purpose of an `if-elif-else` ladder?
2. What is the critical syntax rule in Python for defining a code block?
3. Explain the difference between a `while` loop and a `for` loop.
4. What is an "infinite loop," and how is it usually caused in a `while` loop?
5. What is a function, and what is the "D.R.Y." principle?

WEEK 5: Midterm Test

Topic: Assessment of Weeks 1-4

Introduction:
This week is dedicated to a formal assessment of all concepts covered so far this term.

Scope:

- **Week 1:** Algorithms, Pseudocode, Flowchart symbols and creation.
- **Week 2:** High-level vs. Low-level languages, Compilers vs. Interpreters, Python vs. JavaScript.
- **Week 3:** Variables, Data Types (int, float, str, bool), Type Casting, Operators (Arithmetic, Comparison, Logical).
- **Week 4:** Control Structures (if, elif, else), Loops (while, for), Functions (def, return, parameters).

Format:

- **Section A:** Multiple Choice Questions (Testing definitions and concepts).

PETOA GROUP OF SCHOOLS

- **Section B:** Short Answer Questions (e.g., "Draw the 4 main flowchart symbols," "Explain the difference between == and =.").
- **Section C:** Practical Code/Logic (e.g., "Write the pseudocode for...," "Write a simple Python program to...").

WEEK 6: Data Science Basics

Topic: Data Collection, Storage, and Cleaning

Introduction:
We now shift from general programming to a specific and high-demand application: Data Science. Data Science is the field of using scientific methods to extract knowledge and insights from data.

Subtopic 1: What is Data Science?

PETOA GROUP OF SCHOOLS

- **Definition:** Data Science is an interdisciplinary field that combines:

1. **Computer Science:** (Programming, databases, algorithms).
2. **Statistics & Math:** (Modeling, testing, analysis).
3. **Domain Expertise:** (Knowledge of a specific field like finance or medicine).

- **The Goal:** To turn raw data into meaningful insights, predictions, and business value.
- **The Data Science Lifecycle:** Problem Definition → Data Collection → Data Storage → Data Cleaning → Data Analysis → Modeling → Visualization & Communication.

Subtopic 2: Data Collection

- **Definition:** The process of gathering raw data from various sources.
- **Sources of Data:**

1. **Generated Data:** Transactions (bank records, purchases), Sensors (weather stations, GPS), Surveys/Forms.

2. **Collected Data (from the Web):** Web Scraping (extracting data from websites), APIs (Application Programming Interfaces - a clean way to ask a website for its data).
3. **Existing Data:** Public datasets (data.gov.ng, Kaggle), Databases (internal company records).

Subtopic 3: Data Storage

- **Storage Formats:**

1. **Flat Files (Simple):**

- **CSV (Comma Separated Values):** A simple text file that looks like a spreadsheet. (e.g., `name,age,city`).
- **JSON (JavaScript Object Notation):** A format using key-value pairs, common for web APIs. (e.g., `{"name": "Bello", "age": 25}`).

2. **Databases (Complex):**

- **SQL Databases (Relational):** Organize data into structured tables with rows and columns (like MySQL, PostgreSQL).
- **NoSQL Databases (Non-Relational):** More flexible, used for Big Data (like MongoDB).
- **The GIGO Principle: Garbage In, Garbage Out.** Your analysis is only as good as your data. If your data is "garbage," your results will be "garbage."

Subtopic 4: Data Cleaning (The Most Important Step)

- **Fact:** Data scientists often spend 60-80% of their time just cleaning data.
- **Common Data Cleaning Tasks:**

1. **Handling Missing Values:** Deciding what to do with blank cells (e.g., delete the row, or fill it in with an average).
2. **Removing Duplicates:** Finding and deleting duplicate entries.
3. **Correcting Incorrect Types:** Converting an "Age" column stored as text into numbers.
4. **Standardizing Formats:** Ensuring all entries are consistent (e.g., changing "Lagos", "lagos", and "Lagos State" all to "Lagos").

Evaluation Questions:

1. Define Data Science and list the three fields it combines.
2. What is the "GIGO" principle?
3. List three different sources of data.
4. What is the difference between a CSV file and a JSON file?
5. List three common "data cleaning" tasks.

WEEK 7: Midterm Break

WEEK 8: Big Data and Analytics

Topic: Meaning, Applications, and Tools of Big Data

Introduction:
"Data" is what we handled in Week 6 (e.g., a spreadsheet). "Big Data" is a term for data that is so large, fast, and complex that it cannot be handled with traditional tools like a single computer or Excel.

Subtopic 1: Meaning (The 3 Vs of Big Data)
Big Data is defined by the "3 Vs":

1. **Volume (The Size):** The sheer amount of data. We are talking about Terabytes (TB), Petabytes (PB), and Exabytes (EB).
2. **Velocity (The Speed):** The speed at which new data is created and must be processed (e.g., thousands of Twitter posts per second).
3. **Variety (The Complexity):** The different types of data.

- **Structured:** Clean, organized data (e.g., an Excel sheet).
- **Unstructured:** "Messy" data (e.g., social media posts, videos, images).

Subtopic 2: Applications of Big Data Analytics

- **E-Commerce (Jumia, Konga):** Recommendation engines ("Customers who bought this also bought...") are powered by analyzing the shopping habits of millions of users.
- **Banking & Finance:** Fraud detection systems analyze your normal spending pattern to instantly flag and block an abnormal transaction.
- **Healthcare:** Analyzing millions of patient records to find new drug interactions or predict disease outbreaks.
- **Urban Planning:** Analyzing mobile phone GPS data to understand traffic patterns and plan new roads.
- **Entertainment (Netflix):** Netflix analyzes what you watch to decide what new shows to produce.

Subtopic 3: Tools and Concepts

PETOA GROUP OF SCHOOLS

- **The Problem:** A petabyte of data cannot be stored or processed on one single, massive computer.
- **The Solution (Distributed Computing):**
  - **Concept:** Instead of one supercomputer, you use a network (a "cluster") of hundreds of ordinary computers.
  - **Storage:** The big file is "chopped up" and a small piece is stored on each computer.
  - **Processing:** Each computer solves a small piece of the puzzle, and the results are combined.
- **Key Tools (To Know the Names):**
  - **Hadoop:** The original open-source framework for distributed storage (HDFS) and processing (MapReduce).
  - **Spark:** A newer, faster framework for Big Data analytics.

Evaluation Questions:

1. What are the "3 Vs" that define Big Data? Briefly explain each.
2. What is the difference between "Structured" and "Unstructured" data? Give an example of each.
3. Explain the concept of "Distributed Computing." Why is it necessary for Big Data?
4. Describe a real-world application of Big Data analytics in banking or e-commerce.
5. What is the key difference between the data you might analyze in Excel and "Big Data"?

WEEK 9: Artificial Intelligence (AI)

Topic: Applications in Education, Health, and Business

Introduction:
Artificial Intelligence (AI) is a broad field of computer science dedicated to creating systems that can simulate human intelligence to perform tasks like reasoning, learning, and problem-solving.

PETOA GROUP OF SCHOOLS

Subtopic 1: Understanding AI

- **AI is NOT a Robot:** AI is the "brain" (the software); a robot is the "body" (the hardware). A chatbot is AI, but it has no physical form.
- **Types of AI:**

1. **Weak AI (or Narrow AI):** This is the only type of AI we have today. It is designed and trained to perform one specific, narrow task. (e.g., Siri, a chess program, a spam filter).
2. **Strong AI (or Artificial General Intelligence - AGI):** This is the hypothetical AI that can understand, learn, and apply its intelligence to solve any problem, just like a human. **This does not exist yet.**

Subtopic 2: Applications of (Weak) AI

- **AI in Education:**

1. **Personalized Learning:** Apps like Duolingo adapt to your skill level.
2. **Smart Tutors:** AI-powered chatbots that can answer student questions 24/7.
3. **Automated Grading:** AI systems that can grade tests and simple essays.

- **AI in Healthcare:**

1. **Medical Image Analysis:** AI can detect signs of diseases (like cancer) in X-rays and scans, often better than human doctors.
2. **Diagnosis:** AI "symptom checker" apps help people identify potential illnesses.
3. **Drug Discovery:** AI speeds up the process of discovering new medicines by simulating chemical interactions.

- **AI in Business:**

1. **Customer Service (Chatbots):** AI bots on websites that answer common customer questions.
2. **Supply Chain Optimization:** Using AI to predict product demand and find the most efficient delivery routes.
3. **Human Resources (HR):** AI tools that scan thousands of CVs to find the best candidates.

Evaluation Questions:

1. What is the primary difference between AI (the "brain") and a robot (the "body")?
2. Explain the difference between "Weak AI" and "Strong AI." Which one does not exist yet?
3. Give one example of Weak AI.
4. Describe one major application of AI in healthcare.
5. Describe one major application of AI in business.

WEEK 10: Machine Learning Basics

Topic: Concept and Simple Applications

Introduction:

If AI is the broad goal of simulating human intelligence, Machine Learning (ML) is the primary method for achieving it. ML is a subfield of AI that powers most modern AI applications.

Subtopic 1: Machine Learning vs. Traditional Programming

- **Traditional Programming:**
  - The programmer writes explicit rules (like `if-elif-else`) to tell the computer what to do.
  - **Input:** Data + **Program:** Rules → **Output:** Answer
- **Machine Learning (The New Paradigm):**
  - The programmer feeds the computer data and examples of the answers, and the machine **learns the rules on its own**.
  - **Input:** Data + Example Answers + **Program:** ML Algorithm → **Output:** A "Model" (the set of rules the machine learned)
- **Analogy (Spam Filter):** You cannot write a rule for every single spam word. Instead, you feed an ML algorithm 100,000 emails, label each as "spam" or "not spam", and it learns the patterns of spam on its own.

Subtopic 2: Types of Machine Learning

1. **Supervised Learning (Learning with Labels):**
   - The data is labeled with the correct answers.
   - **Goal:** To make predictions.
   - **Examples:**
     - **Classification:** Predicting a category (e.g., "Spam/Not Spam", "Cat/Dog").
     - **Regression:** Predicting a number (e.g., "What will be the price of this house?").
2. **Unsupervised Learning (Learning without Labels):**
   - The data has no labels. You don't know the "right" answer.
   - **Goal:** To find hidden patterns or structures in the data.

- **Examples:**

- **Clustering:** Automatically grouping data points (e.g., grouping customers into "high-spenders" and "bargain-hunters").
- **Anomaly Detection:** Finding data points that don't fit any pattern (e.g., a weird sensor reading that could be fraud).

Evaluation Questions:

1. What is the key difference between traditional programming and machine learning?
2. In machine learning, what is a "Model"?
3. What is the difference between Supervised Learning and Unsupervised Learning?
4. What kind of ML task is a spam filter? (Supervised/Classification, Supervised/Regression, or Unsupervised/Clustering?)
5. What kind of ML task is "grouping customers by their secret shopping habits"?

PETOA GROUP OF SCHOOLS

WEEK 11: Revision of Programming Fundamentals

Topic: Consolidation of Weeks 1-4

Introduction:
This week is dedicated to a comprehensive review of all programming topics covered from Week 1 to Week 4.

Revision Scope (Weeks 1-4):

- **Week 1 (Algorithms & Flowcharts):**

- Definition and properties of algorithms.
- Writing pseudocode.
- Drawing and interpreting flowcharts (Terminator, Process, Input/Output, Decision symbols).

- **Week 2 (Programming Languages):**
  - High-level vs. Low-level languages.
  - Compilers vs. Interpreters.
  - Key features of Python and JavaScript.
- **Week 3 (Programming Basics):**
  - Variables and naming conventions.
  - Data Types (`int`, `float`, `str`, `bool`).
  - Type casting (`int()`, `float()`, `str()`).
  - Arithmetic, Comparison, and Logical operators.
- **Week 4 (Control Structures):**
  - Conditional statements (`if`, `elif`, `else`).
  - Loops (`while` and `for`).
  - Functions (`def`, parameters, `return`).

Activity:

- Solve mixed problems combining all concepts.
- Review the Week 5 Midterm Test.
- Practice writing short Python scripts.

PETOA GROUP OF SCHOOLS

WEEK 12: Revision of Data Science and AI/ML

Topic: Consolidation of Weeks 6, 8, 9, 10

Introduction:
This week is dedicated to a comprehensive review of all Data Science, Big Data, AI, and Machine Learning topics.

Revision Scope (Weeks 6, 8, 9, 10):

- **Week 6 (Data Science Basics):**
  - The Data Science Lifecycle.
  - Data Collection methods (APIs, web scraping).
  - Data Storage (CSV, JSON, Databases).
  - Data Cleaning (GIGO, handling missing values).
- **Week 8 (Big Data):**
  - The 3 Vs of Big Data (Volume, Velocity, Variety).
  - Real-world applications (Fraud Detection, Recommendations).
  - Distributed Computing (Hadoop, Spark).
- **Week 9 (Artificial Intelligence):**
  - Weak (Narrow) AI vs. Strong (General) AI.
  - Applications in Education, Healthcare, and Business.
- **Week 10 (Machine Learning):**
  - Traditional Programming vs. Machine Learning.
  - Supervised Learning (Classification, Regression) vs. Unsupervised Learning (Clustering).

Activity:

- Class discussion on the ethical implications of AI and Big Data.
- Group work: Propose a simple ML solution to a school or community problem.

WEEK 13: Final Examinations

Topic: Second Term Examination

This week is for your final examinations on all topics covered in the Second Term. Read each question carefully and do your best.

Good luck!

PETOA GROUP OF SCHOOLS

PETOA
GROUP OF SCHOOLS